

LESSON 6 Markov Decision Process and Reinforcement Learning

In Lesson 5, we learned about Markov chains. Markov chains are potentially useful for modeling systems evolving in time, for example, they provide a simple model for a **time-series** e.g. the mini web. Markov chains are also used in a **reinforcement learning** in the form of a **Markov decision process** (MDP), which has a range of applications including autonomous flight, robot motion, network routing etc. This is the topic of this lesson.

6.1 Markov Decision Process

We will start with a simple example. Suppose a robot ⌘ can be in any of the 11 states excluding the gray square. Moreover, the robot can move Left $\leftarrow$, Right $\rightarrow$, Up $\uparrow$, Down $\downarrow$, so in the MDP, in addition to the set S of 11 states, there is a set A of 4 **actions**.

1	2	3	4				+1
5		6	7				-1
8	9	10	11			⌘	

There is a set of **transition probabilities** $P_{sa}(s')$ which depend on the action a , giving probability of moving from state s to state s' . For example, in its initial position 10, suppose that if the robot moves $\uparrow$, it has probability 0.8 moving to state 6, probability 0.1 moving to state 9 or state 11, then $P_{10\uparrow}(6) = 0.8$, $P_{10\uparrow}(9) = P_{10\uparrow}(11) = 0.1$, and $P_{10\uparrow}(s') = 0$ for all other s' .

There is a **reward function** $R(s)$. Here, let $R(4) = 1$, $R(7) = -1$, and $R(s) = 0$ for other states s . If the robot has perfect locomotion, then starting at state 10, it should simply take the series of actions $\uparrow \uparrow \rightarrow$ to get to +1. However, because of its imperfect movement, after moving up, it may end up at -1 inadvertently. Probability has entered the process

$$s_0 \xrightarrow{a_0} s_1 \xrightarrow{a_1} s_2 \xrightarrow{a_2} s_3 \xrightarrow{a_3} \dots$$

The transition probabilities and the reward function may be given or “learned” from data:

$$P_{sa}(s') = \frac{\# \text{ times we took action } a \text{ in state } s \text{ and got to state } s'}{\# \text{ times we took action } a \text{ in state } s}$$

Reward can be similarly learned.

6.2 Bellman Equation

Our goal is to find a **policy** $\pi: S \rightarrow A$ which will result in a series of actions maximizing the **value function** or **expected total payoff** over time:

$$V^\pi(s_0) = E[R(s_0) + \gamma R(s_1) + \gamma^2 R(s_2) + \dots]$$

There is a **discount factor** $\gamma < 1$ (“time is money”), and we want to collect positive rewards as soon as possible. We can rewrite:

$$V^\pi(s_0) = R(s_0) + \gamma E[R(s_1) + \gamma R(s_2) + \gamma^2 R(s_3) + \dots]$$

Note that $E[R(s_1) + \gamma R(s_2) + \gamma^2 R(s_3) + \dots] = V^\pi(s_1)$ and given initial state s_0 , we have that $s_1 = s'$ with transition probability $P_{s_0\pi(s_0)}(s')$, so

$$V^\pi(s_1) = \sum_{s' \in S} P_{s_0\pi(s_0)}(s') V^\pi(s')$$

and we have the **Bellman equation** for fixed policy π and initial state s_0 :

$$V^\pi(s_0) = R(s_0) + \gamma \sum_{s' \in \mathcal{S}} P_{s_0 \pi(s_0)}(s') V^\pi(s')$$

For a MDP with n states, for a fixed policy, we may write down an equation for each of the n possible s_0 's, so the Bellman equation is a system of n linear equations in n variables $V^\pi(s_0)$'s, which we can solve to find $V^\pi(s_0)$ for each of the n possible s_0 's.

6.3 Optimal Policy and Policy Iteration

The goal is to find the **optimal policy** $\pi^*(s_0)$ such that $V^*(s_0) = V^{\pi^*}(s_0) \geq V^\pi(s_0)$ for every π , for each initial state s_0 , that is,

$$V^*(s_0) = \max_{\pi} V^\pi(s_0), \quad \pi^*(s_0) = \arg \max_{\pi} V^\pi(s_0)$$

With some manipulation, we get the Bellman equation for the optimal value function $V^*(s_0)$:

$$\begin{aligned} V^*(s_0) &= \max_{\pi} V^\pi(s_0) \\ &= \max_{\pi} \left[R(s_0) + \gamma \sum_{s' \in \mathcal{S}} P_{s_0 \pi(s_0)}(s') V^\pi(s') \right] \\ &= R(s_0) + \gamma \left[\max_{\pi} \sum_{s' \in \mathcal{S}} P_{s_0 \pi(s_0)}(s') V^\pi(s') \right] \\ &= R(s_0) + \gamma \sum_{s' \in \mathcal{S}} P_{s_0 \pi^*(s_0)}(s') V^{\pi^*}(s') \\ &= R(s_0) + \gamma \sum_{s' \in \mathcal{S}} P_{s_0 \pi^*(s_0)}(s') V^*(s') \end{aligned}$$

It is now useful to rewrite the optimal policy as actions which maximize reward:

$$\max_{\pi} \sum_{s' \in \mathcal{S}} P_{s_0 \pi(s_0)}(s') V^\pi(s') = \sum_{s' \in \mathcal{S}} P_{s_0 \pi^*(s_0)}(s') V^*(s') = \max_{a \in \mathcal{A}} \sum_{s' \in \mathcal{S}} P_{s_0 a}(s') V^*(s')$$

This equation says that maximizing the future reward $\sum_{s' \in \mathcal{S}} P_{s_0 \pi(s_0)}(s') V^\pi(s')$ entails picking the action $\pi(s_0) = a$ that will maximize $\sum_{s' \in \mathcal{S}} P_{s_0 a}(s') V^*(s')$, where $V^*(s')$ is the future optimal value starting in the next state $s' = a(s_0)$.

Interestingly, the optimal policy $\pi^*(s_0)$ is independent of the initial state s_0 , but this is not surprising given that the Markov process is completely determined by the one-time-step transition probabilities: no matter what the history, what matters for the present action and the next state is the current state. There are two iterative algorithms which find the optimal policy. We will only describe **Policy Iteration** (the other is Value Iteration).

We choose an initial policy π randomly and solve the Bellman equation to find $V^\pi(s)$. For each state s , the action in the new policy π' is $\pi'(s) = \arg \max_{a \in \mathcal{A}} \sum_{s' \in \mathcal{S}} P_{sa}(s') V^\pi(s')$. We repeat until convergence, that is, the policy no longer changes.

To conclude: reinforcement learning is the idea that learning is through a reward system and experience, without explicit supervision, and MDP provides the basic formalism.

Question: is reinforcement learning the same as Q-learning?